Computer Programming 143 – Lecture 28
Dynamic Data Structures II

Faculty of Engineering



Copyright & Disclaimer

Copyright

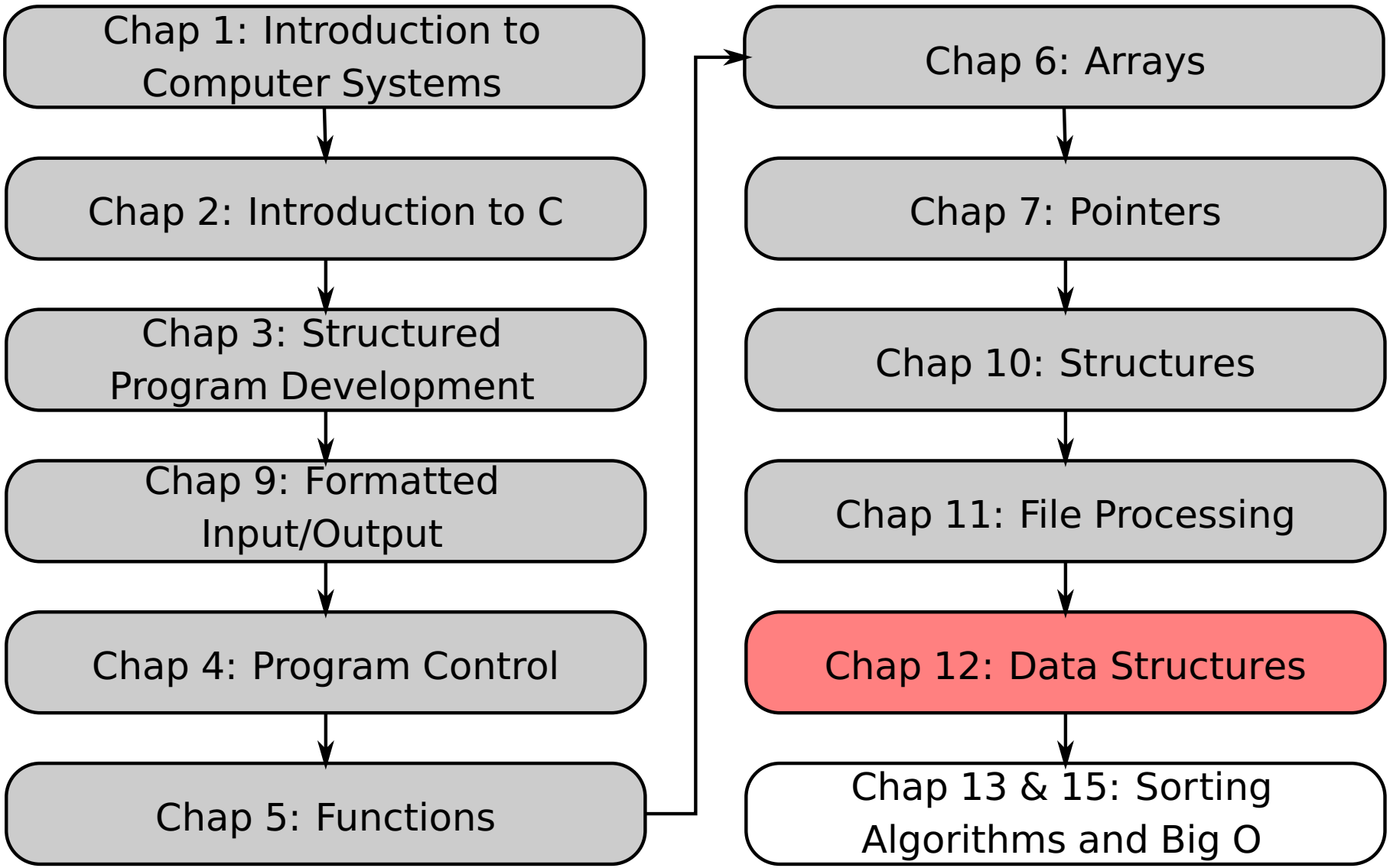
Copyright © 2026 Stellenbosch University
All rights reserved

Disclaimer

This content is provided without warranty or representation of any kind. The use of the content is entirely at your own risk and Stellenbosch University (SU) will have no liability directly or indirectly as a result of this content.

The content must not be assumed to provide complete coverage of the particular study material. Content may be removed or changed without notice.

Module Overview



Lecture Overview

1. Stacks (12.5)

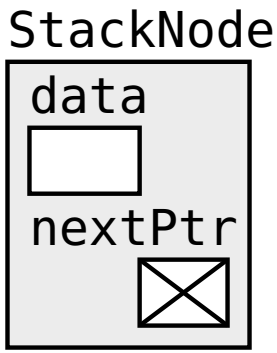
2. Queues (12.6)

12.5 Stacks

Stacks

- Constrained version of a linked list
  - Is a row/line of self-referential structures
- Elements may only be inserted or deleted from the top (front) of a stack
  - A stack is therefore a **last-in-first-out structure (LIFO)**

```
struct stackNode {
    int data;
    struct stackNode *nextPtr;
};
typedef struct stackNode StackNode;
```

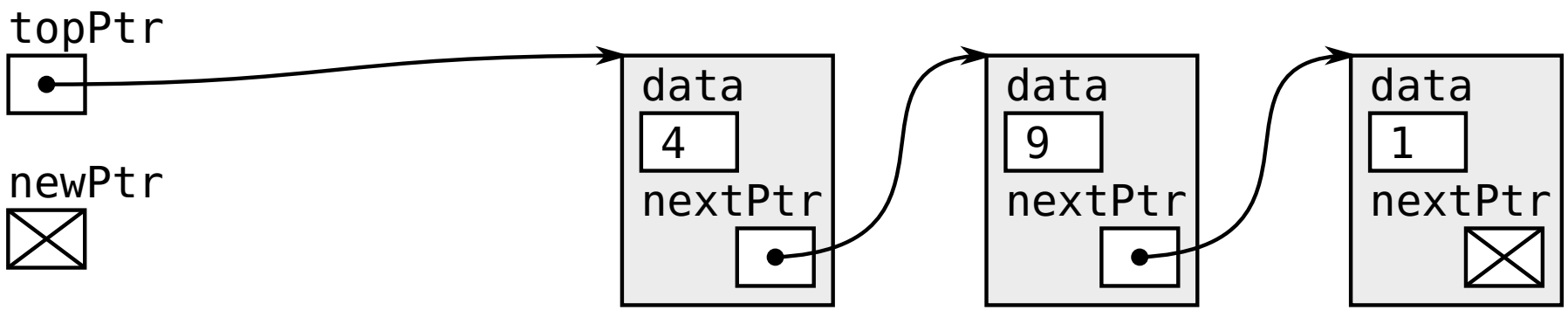


Use of stacks

- Function calls – store of return address and automatic local variables
- Used by compilers when evaluating expressions

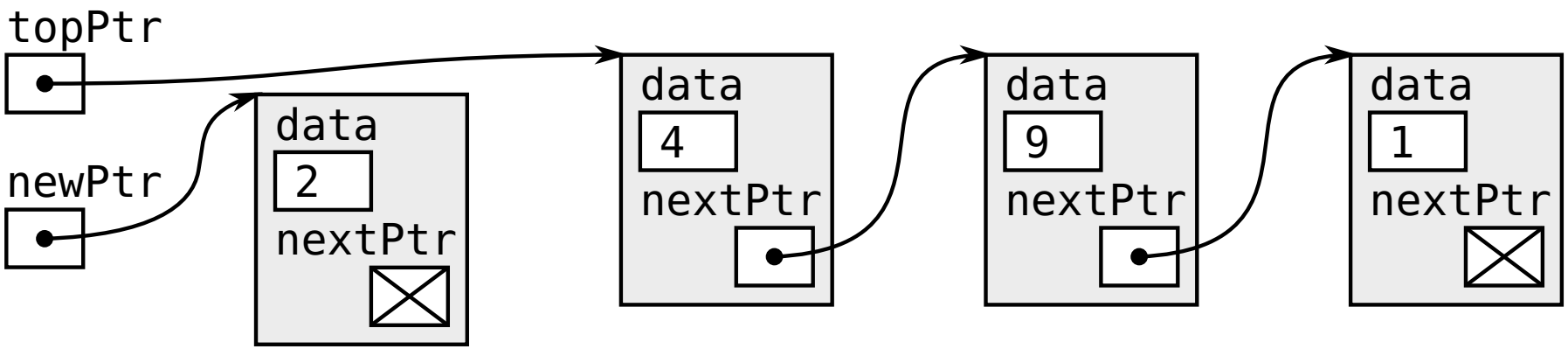
12.5 Insertion in Stacks (“Push”)

```
newPtr = malloc( sizeof( StackNode ) );
newPtr->data = value;
newPtr->nextPtr = topPtr;
topPtr = newPtr;
```



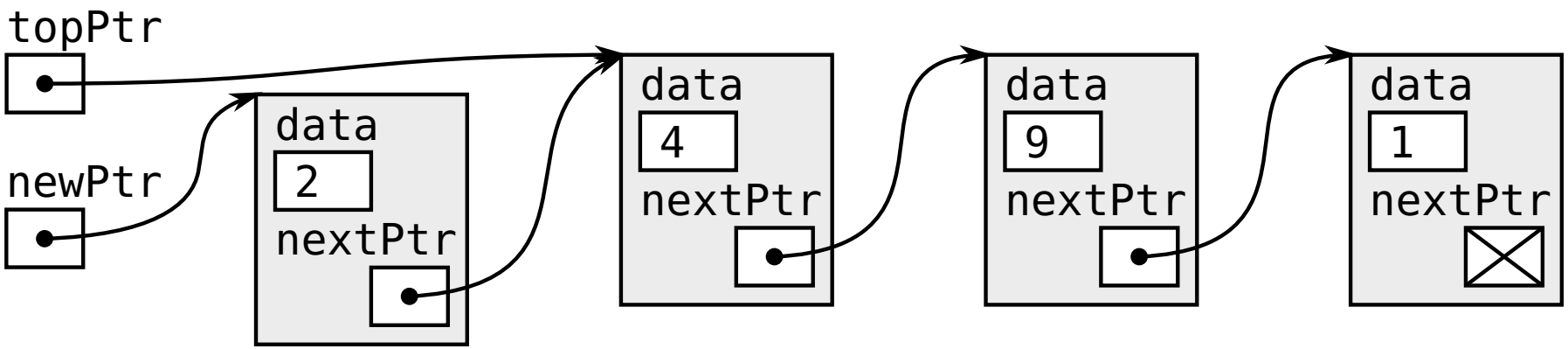
12.5 Insertion in Stacks (“Push”)

```
newPtr = malloc( sizeof( StackNode ) );
newPtr->data = value;
newPtr->nextPtr = topPtr;
topPtr = newPtr;
```



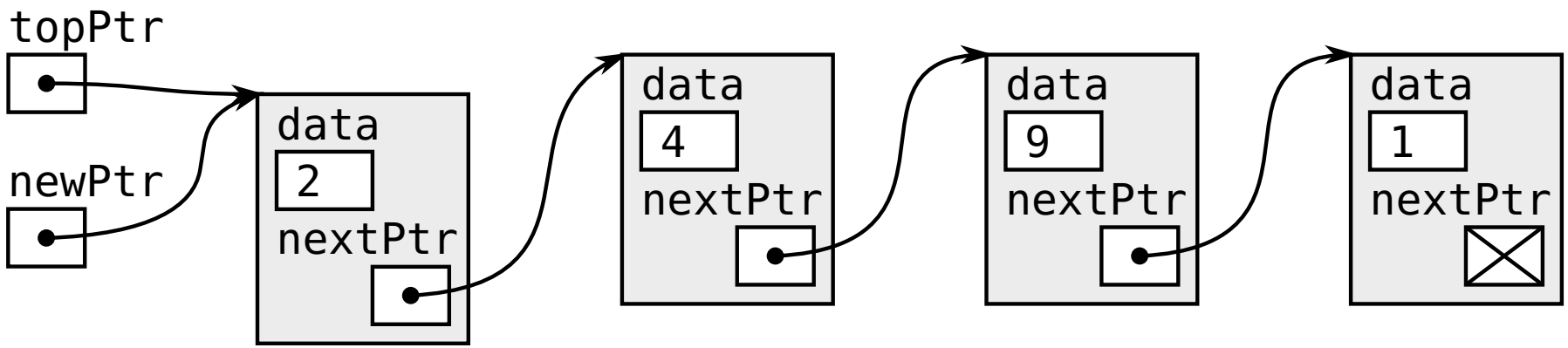
12.5 Insertion in Stacks (“Push”)

```
newPtr = malloc( sizeof( StackNode ) );
newPtr->data = value;
newPtr->nextPtr = topPtr;
topPtr = newPtr;
```



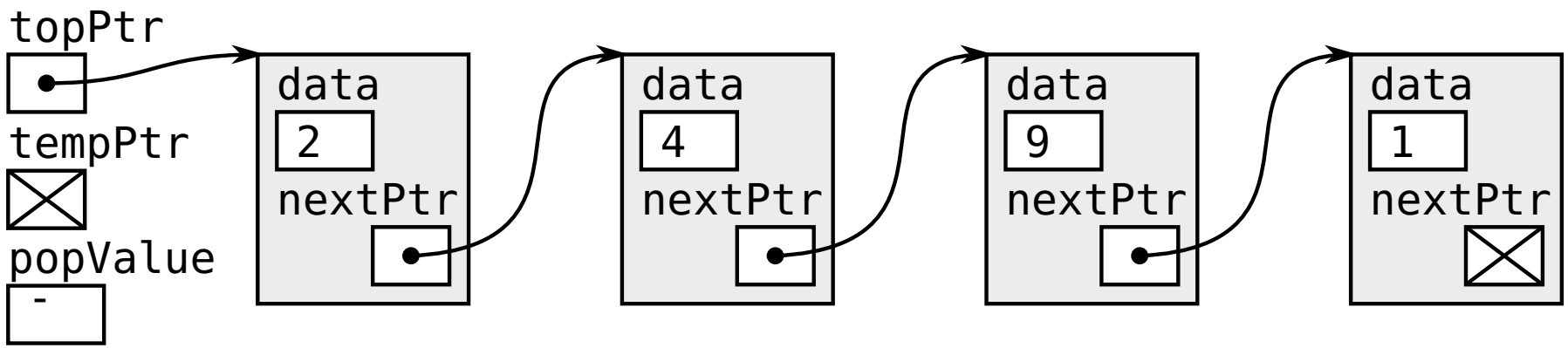
12.5 Insertion in Stacks (“Push”)

```
newPtr = malloc( sizeof( StackNode ) );
newPtr->data = value;
newPtr->nextPtr = topPtr;
topPtr = newPtr;
```



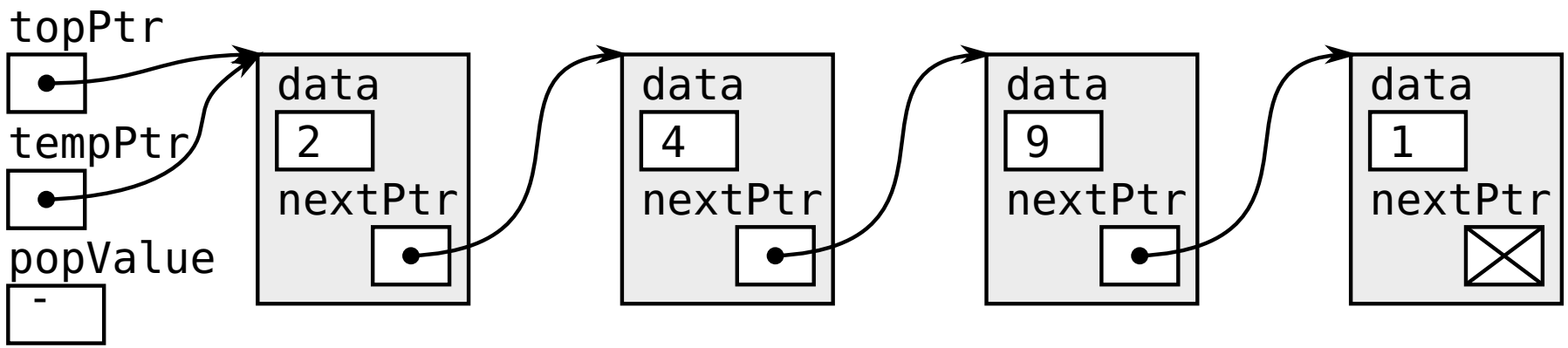
12.5 Deletion in Stacks (“Pop”)

```
tempPtr = topPtr;
popValue = topPtr->data;
topPtr = topPtr->nextPtr;
free( tempPtr );
```



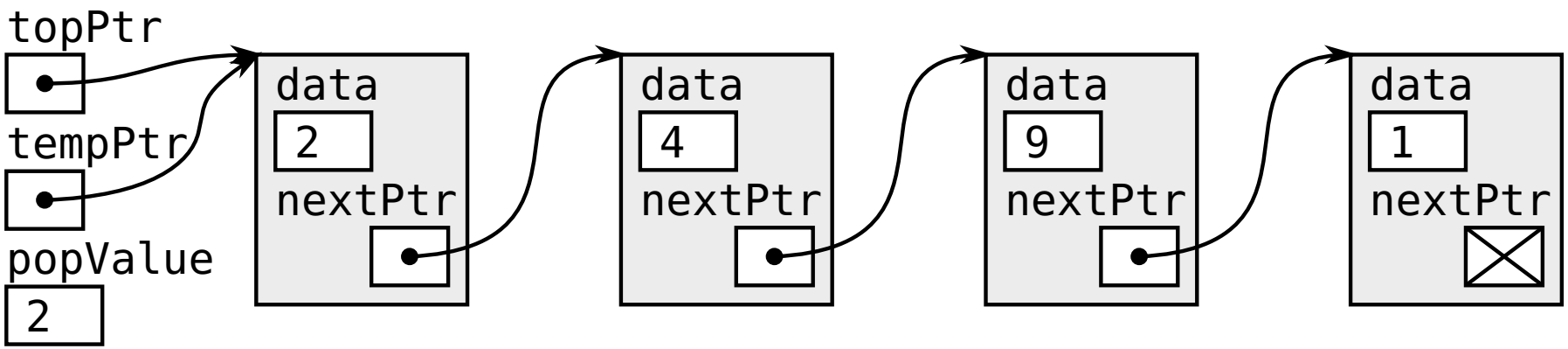
12.5 Deletion in Stacks (“Pop”)

```
tempPtr = topPtr;
popValue = topPtr->data;
topPtr = topPtr->nextPtr;
free( tempPtr );
```



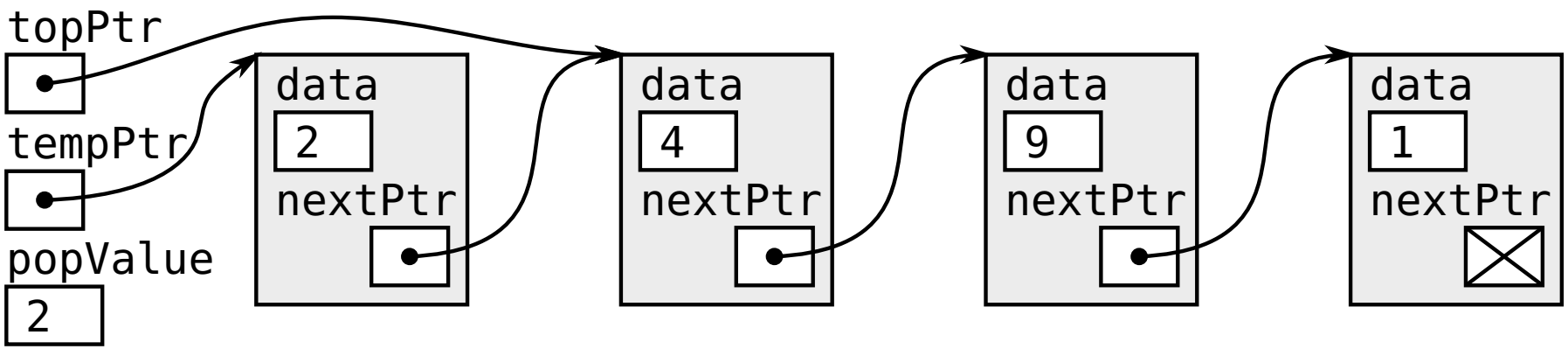
12.5 Deletion in Stacks (“Pop”)

```
tempPtr = topPtr;
popValue = topPtr->data;
topPtr = topPtr->nextPtr;
free( tempPtr );
```



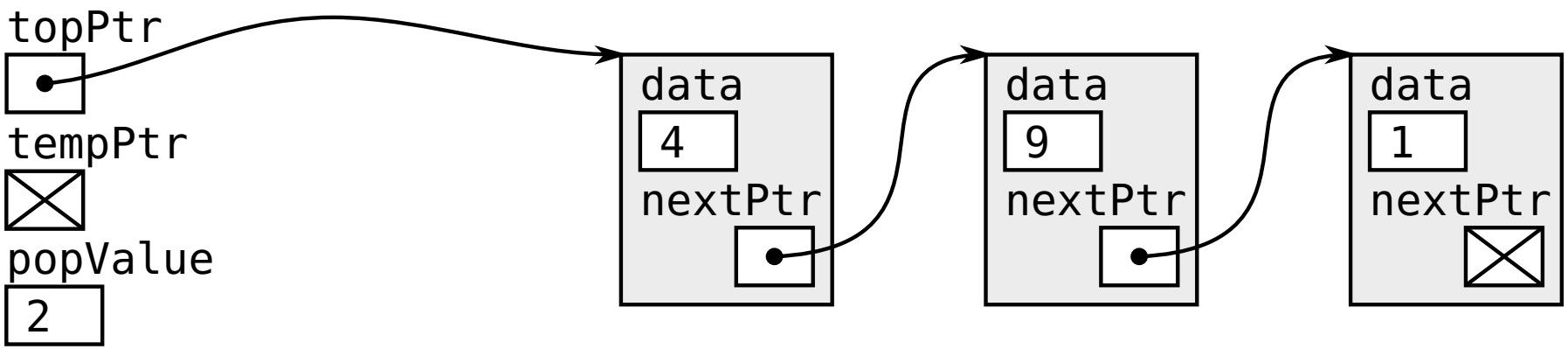
12.5 Deletion in Stacks (“Pop”)

```
tempPtr = topPtr;
popValue = topPtr->data;
topPtr = topPtr->nextPtr;
free( tempPtr );
```



12.5 Deletion in Stacks (“Pop”)

```
tempPtr = topPtr;
popValue = topPtr->data;
topPtr = topPtr->nextPtr;
free( tempPtr );
```

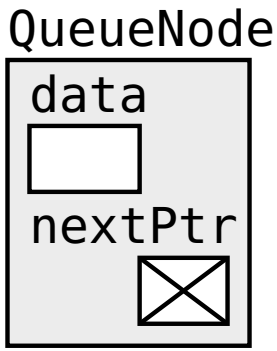


12.6 Queues

Queues

- Constrained version of a linked list
  - Is a row/line of self-referential structures
- Elements may only be deleted from the front (head) of a stack and inserted at the back (tail)
  - A queue is therefore a **first-in-first-out structure (FIFO)**

```
struct queueNode {
    int data;
    struct queueNode *nextPtr;
};
typedef struct queueNode QueueNode;
```

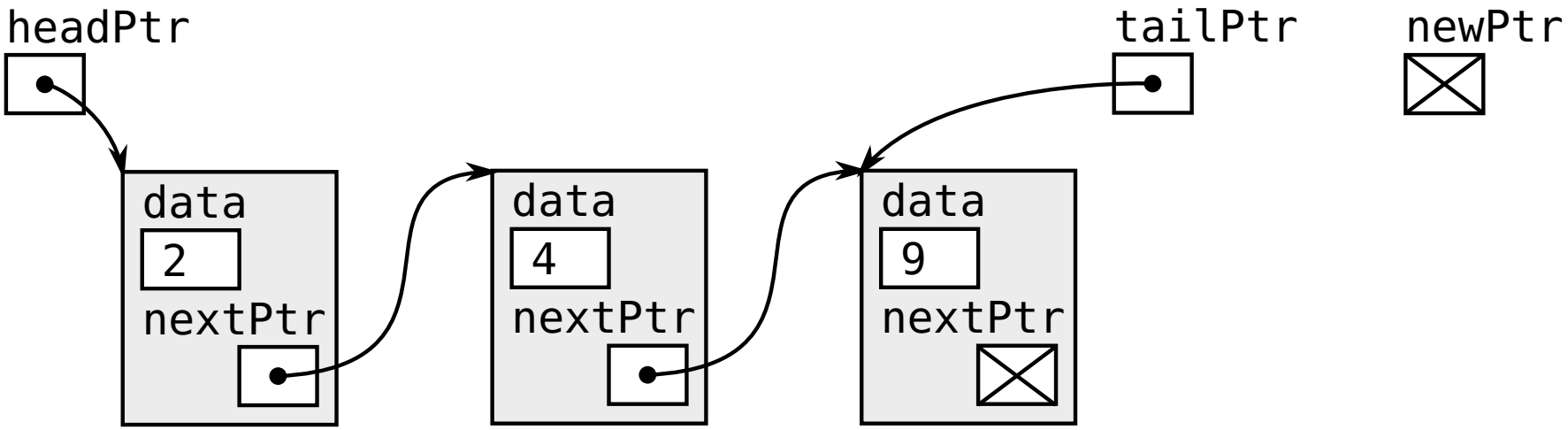


Use of Queues

- Supporting print spooling
- Routing of data packets in computer systems
- Buffers

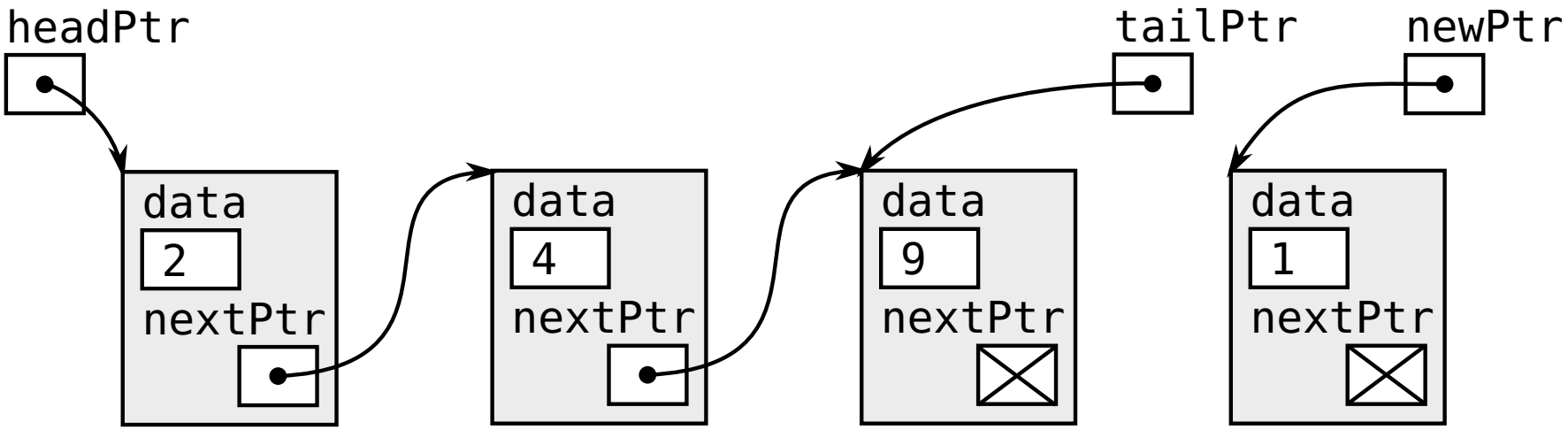
12.6 Insertion in Queues (“Enqueue”)

```
newPtr = malloc( sizeof( QueueNode ) );
newPtr->data = value;
newPtr->nextPtr = NULL;
tailPtr->nextPtr = newPtr;
tailPtr = newPtr;
```



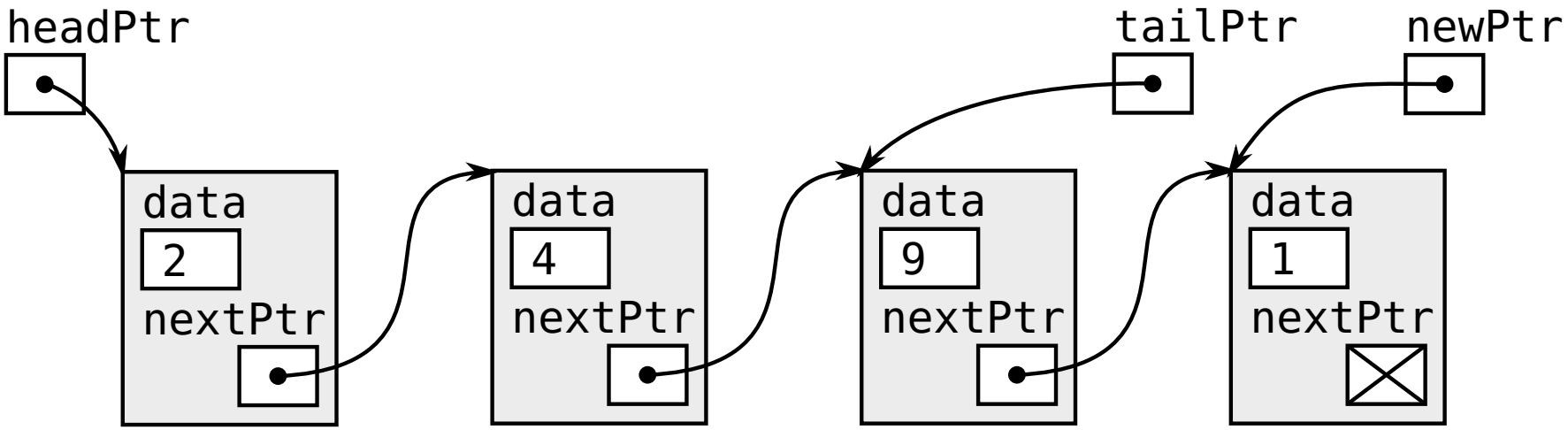
12.6 Insertion in Queues (“Enqueue”)

```
newPtr = malloc( sizeof( QueueNode ) );
newPtr->data = value;
newPtr->nextPtr = NULL;
tailPtr->nextPtr = newPtr;
tailPtr = newPtr;
```



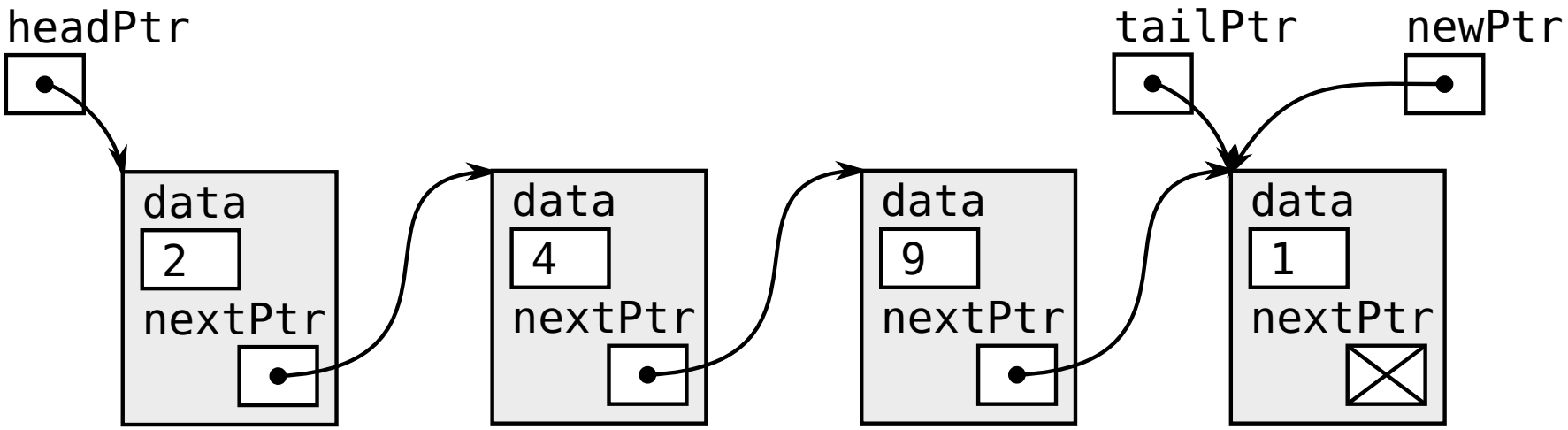
12.6 Insertion in Queues (“Enqueue”)

```
newPtr = malloc( sizeof( QueueNode ) );
newPtr->data = value;
newPtr->nextPtr = NULL;
tailPtr->nextPtr = newPtr;
tailPtr = newPtr;
```



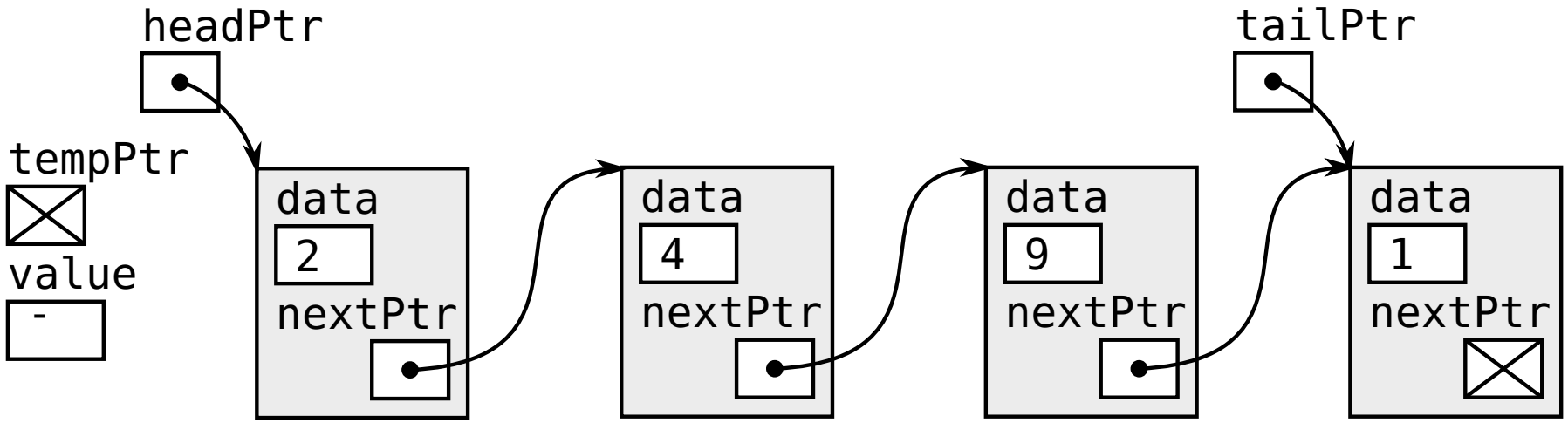
12.6 Insertion in Queues (“Enqueue”)

```
newPtr = malloc( sizeof( QueueNode ) );
newPtr->data = value;
newPtr->nextPtr = NULL;
tailPtr->nextPtr = newPtr;
tailPtr = newPtr;
```



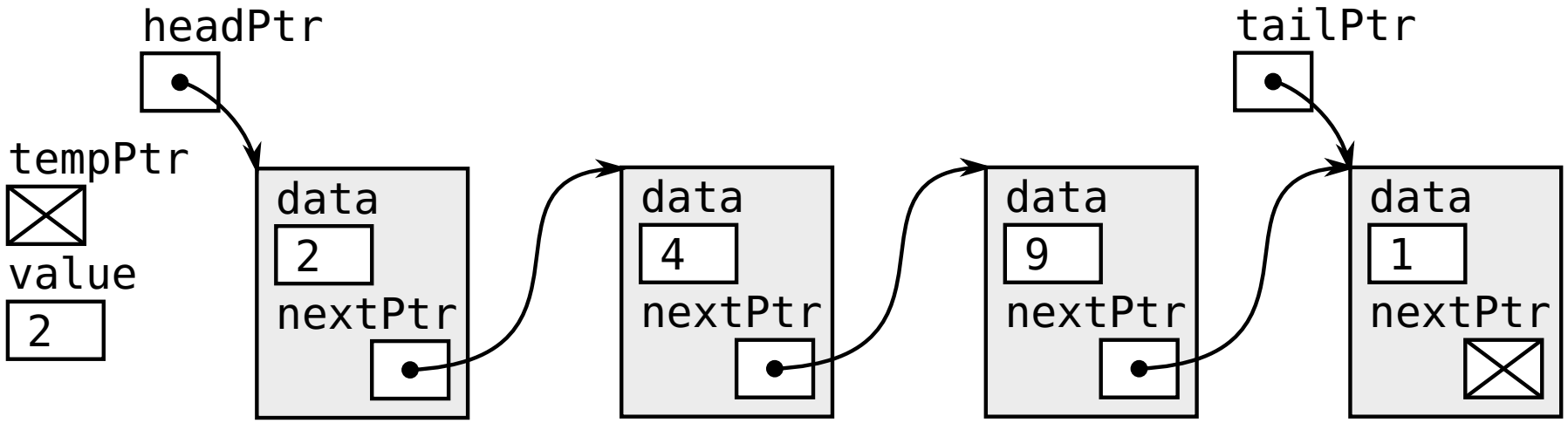
12.6 Deletion in Queues (“Dequeue”)

```
value = headPtr->data;
tempPtr = headPtr;
headPtr = headPtr->nextPtr;
free( tempPtr );
```



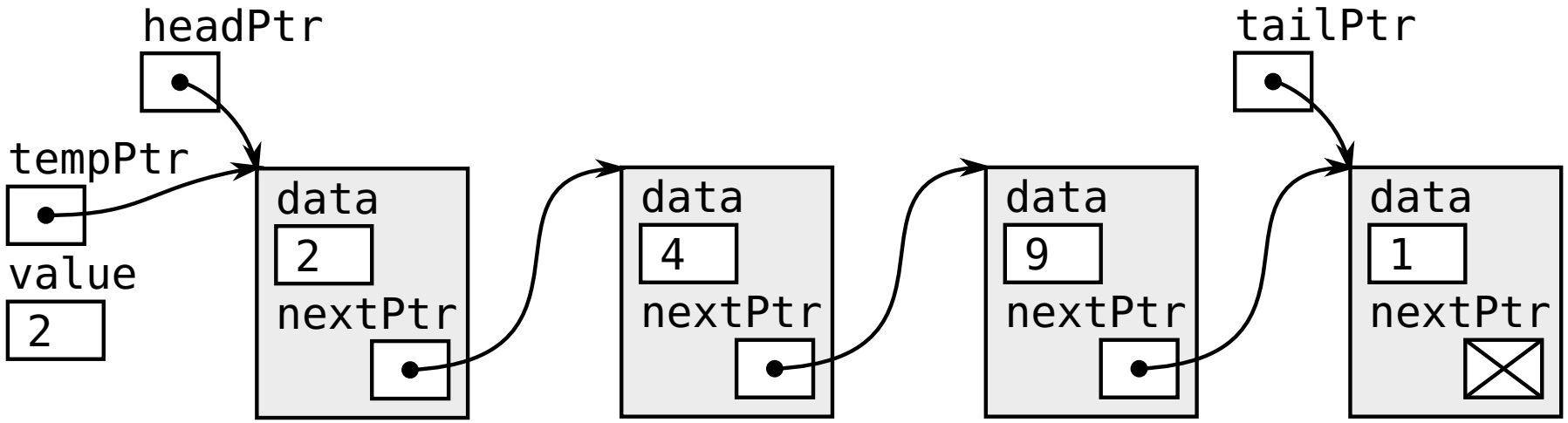
12.6 Deletion in Queues (“Dequeue”)

```
value = headPtr->data;
tempPtr = headPtr;
headPtr = headPtr->nextPtr;
free( tempPtr );
```



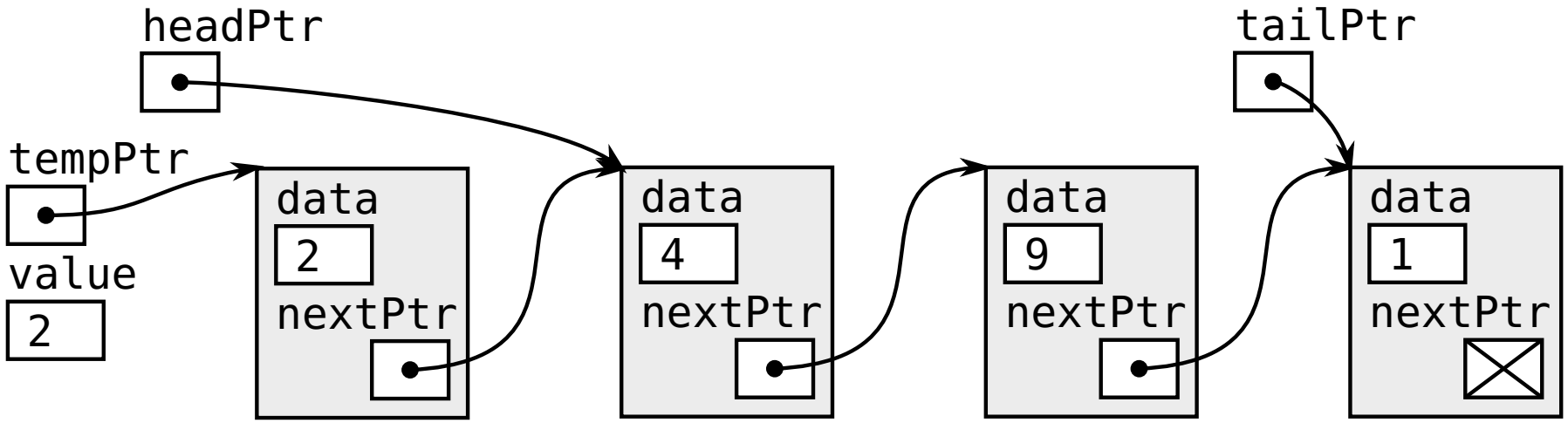
12.6 Deletion in Queues (“Dequeue”)

```
value = headPtr->data;
tempPtr = headPtr;
headPtr = headPtr->nextPtr;
free( tempPtr );
```



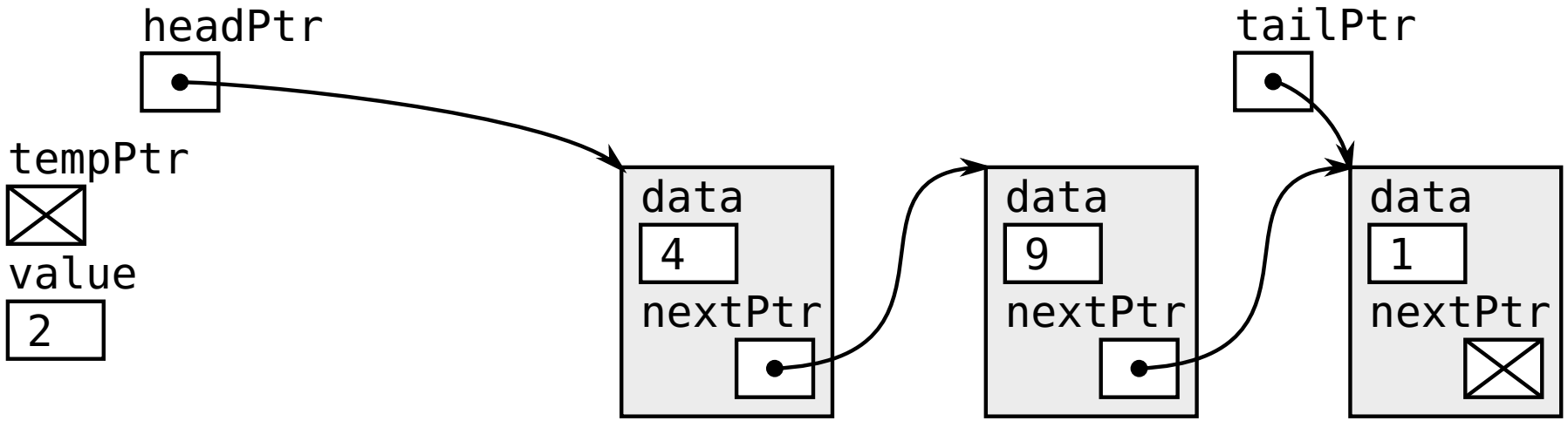
12.6 Deletion in Queues (“Dequeue”)

```
value = headPtr->data;
tempPtr = headPtr;
headPtr = headPtr->nextPtr;
free( tempPtr );
```



12.6 Deletion in Queues (“Dequeue”)

```
value = headPtr->data;
tempPtr = headPtr;
headPtr = headPtr->nextPtr;
free( tempPtr );
```



Perspective

Today

Dynamic Data Structures II

- Stacks
- Queues

Next lecture

Dynamic Data Structures III

- Trees

Homework

1. Study Sections 12.5-12.6 in Deitel & Deitel
2. Do Self Review Exercises 12.2, 12.3 in Deitel & Deitel
3. Do Exercise 12.8 in Deitel & Deitel